

SQL Server

Advanced Performance

Contents

- Introduction
- Clustered & Non-clustered Indexes
- Covering Indexes
- Concatenated Indexes
- Working with indexes
- Rules of thumb
- Quiz
- Materialized views
- Index statistics
- Storage & partitions

Contents

- **Introduction**
- Clustered & Non-clustered Indexes
- Covering Indexes
- Concatenated Indexes
- Working with indexes
- Rules of thumb
- Quiz
- Materialized views
- Index statistics
- Storage & partitions

Is performance still relevant?

Because:

Transistor density on a manufactured semiconductor doubles about every 18 months.

Moore's law

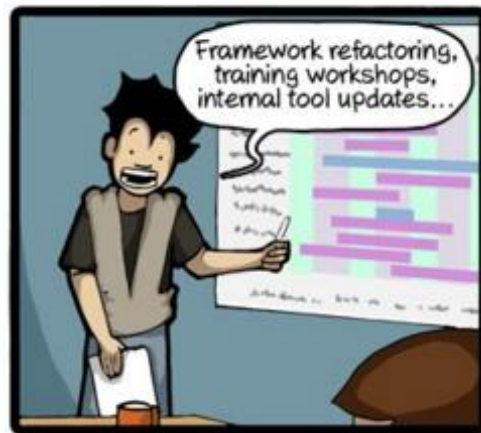
(no longer valid since 2016?)

But:

Software gets slower faster than hardware gets faster

Wirth's law

Anyway...



Often indexes offer the solution



Space allocation by SQL Server

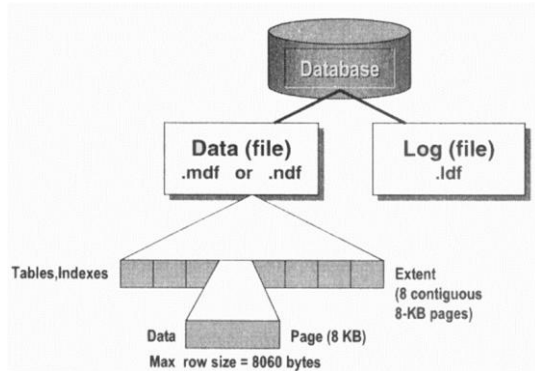
Database name: xtreme

Owner:

☒ Use full-text indexing

Database files:

Logical Name	File Type	Filegroup	Initial Size (MB)
Xtreme_Data	ROWS Data	PRIMARY	14
Xtreme_Log	LOG	Not Applicable	32



- SQL Server uses random access files
- Space allocation in *extents* and *pages*
- Page = 8 kB block of contiguous space
- Extent = 8 logical consecutive pages.
 - uniform extents: for one db object
 - mixed extents: can be shared by 8 db objects (=tables, indexes)
- New table or index: allocation in mixed extent
- Extension > 8 pages: in uniform extent

Contents

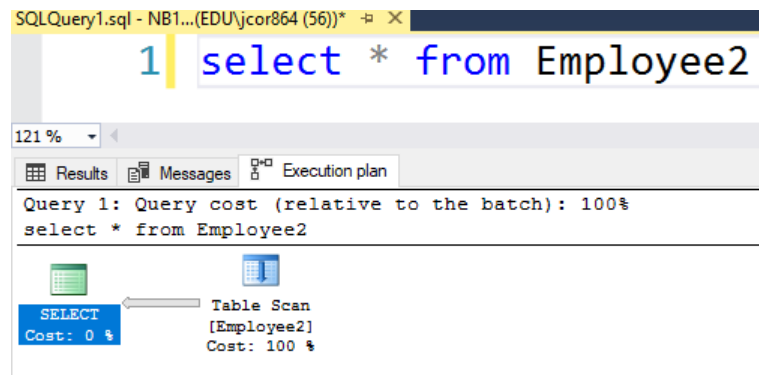
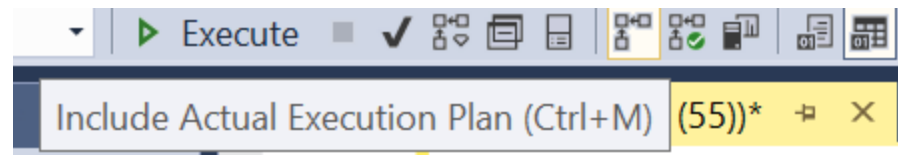
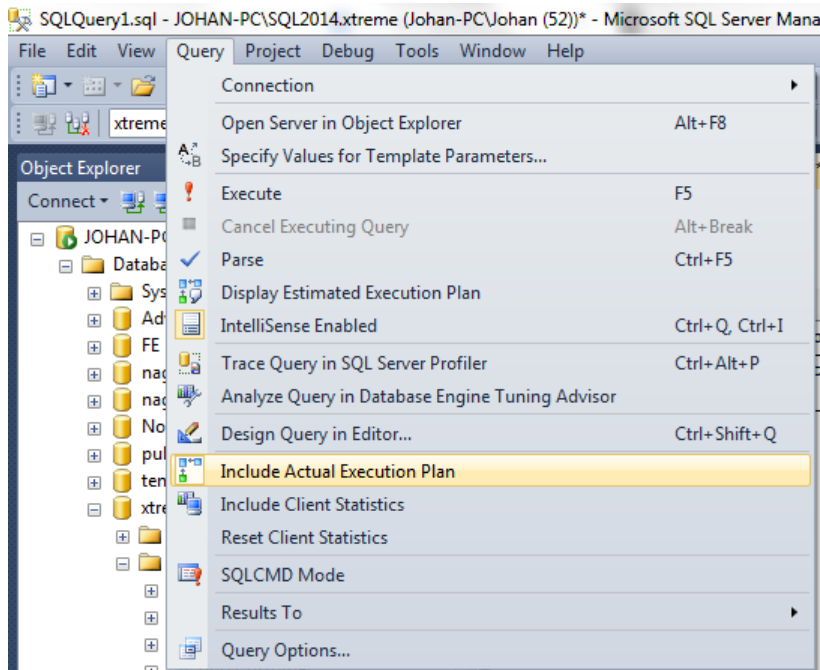
- Introduction
- **Clustered & Non-clustered Indexes**
- Covering Indexes
- Concatenated Indexes
- Working with indexes
- Rules of thumb
- Quiz
- Materialized views
- Index statistics
- Storage & partitions

Table scan

- Heap: unordered collection of data-pages without clustered index (see below) = default storage of a table
- Access via Index Allocation Map (IAM)
- Table scan: if a query fetches all pages of the table → **always to avoid!**
- Other performance issues with heap:
 - fragmentation: table is scattered over several, non-consecutive pages
 - forward pointers: if a variable length row (e.g. varchar fields) becomes longer upon update, a forward pointer to another page is added.
→ table scan even slower

Does my query cause a table scan?

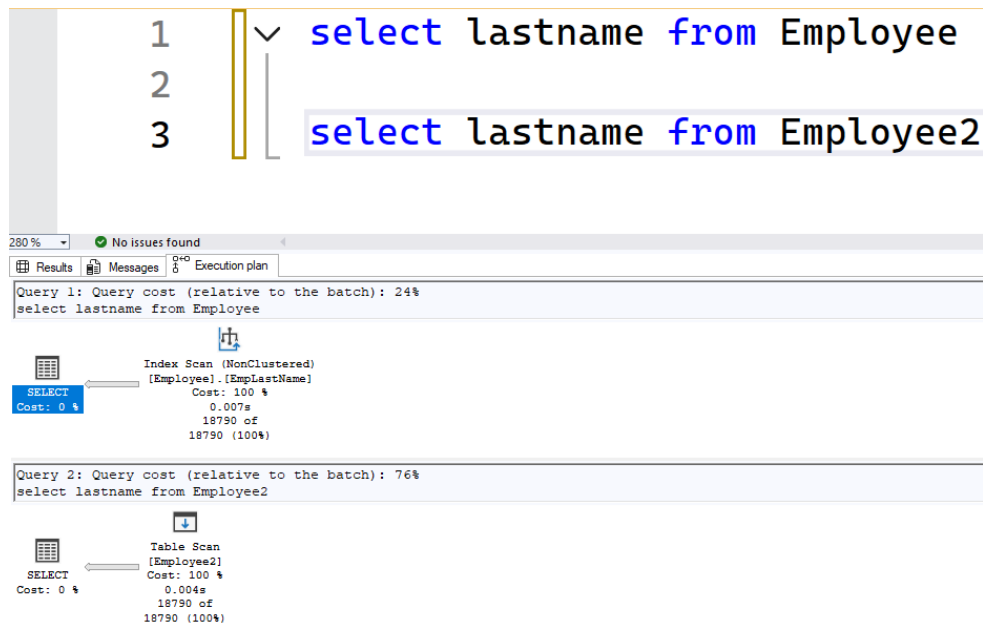
Examine the Execution Plan of the query (db xtreme + script EmployeeIdx.sql)



Compare 2 queries

(db xtreme + script EmployeeIdx)

- Execute the 2 queries together (select both + Execute!)
- Table Employee2 is a copy of Employee, but without indexes
- Query on Employee2 takes 3x longer!



What is the difference? Indexes!

- **What?**
 - ordered structure imposed on records from a table
 - Fast access through tree structure (B-tree = balanced tree)
- **Why?**
 - can speed up data retrieval
 - can force unicity of rows
- **Why not ?**
 - indexes consume storage (overhead)
 - Indexes can slow down updates, deletes and inserts because indexes have to be updated too

Indexes: book analogy

INDEX

SYMBOLS

2PC, 89
?, :var, @var (see bind parameter)

A

Access Predicate, 44
access predicates
 recognizing in execution plans
 Oracle, 170
 PostgreSQL, 177
 SQL Server, 185
adaptive cursor sharing (Oracle), 75
auto parameterization (SQL Server), 39

B

B-tree (balanced search tree), 4
between, 44
bind parameter, 32
 contraindications
 histograms, 34
 LIKE filters, 47
 partitions, 35
 for execution plan caching, 32
 type safety, 66
bind peeking (Oracle), 75
bitmap index, 50
Bitmap Index Scan (PostgreSQL), 175
Brewer's CAP theorem, 89

C

CAP theorem, 89
cardinality estimate, 27
CBO (see optimizer, cost based)
clustered index, 122
 transform to SQL Server heap table, 127
clustering factor, 21, 114
 automatically optimized, 133
clustering key, 123
collation, 24
commit
 deferrable constraints, 11
 implicit for truncate table, 163
 two phase, 89
compiling, 18
computed columns (SQL Server), 27
constraint
 deferrable, 11
 NOT NULL, 56
cost value, 18

count(*)
 often as index-only scan, 120
 Oracle requires NOT NULL constraint, 57
COUNT STOPKEY, 145
cursor sharing (Oracle), 39

D

data transport object (DTO), 105
DATE
 efficiently working with, 62
DBMS_XPLAN, 167
DEALLOCATE, 174
DEFERRABLE constraint, 11
DETERMINISTIC (Oracle), 30
distinct, 97
distinct()
 in JPA and Hibernate, 97
DML, 159
doubly linked list, 2
dynamic-update (Hibernate), 164

E

eager fetching, 96
eventual consistency, 89
execution plan, 10, 165
 cache, 32, 75
 creating
 MySQL, 188
 Oracle, 166
 PostgreSQL, 172
 SQL Server, 180
 operations
 MySQL, 188
 Oracle, 167
 PostgreSQL, 174
 SQL Server, 182
explain
 MySQL, 188
 Oracle, 166
 PostgreSQL, 172

F

FBI (see index, function-based)
FETCH ALL PROPERTIES (HQL), 105
fetch first, 144
filter predicates
 effects (chart), 81
 recognizing in execution plans
 Oracle, 170
 PostgreSQL, 177
 SQL Server, 185

full table scan, 13
 A11 (MySQL), 189
 Seq Scan (PostgreSQL), 174
 TABLE ACCESS FULL (Oracle), 168
 Table Scan (SQL Server), 183
functions, 24
 deterministic, 29
 in partial indexes, 52
 window, 156

G

group by, 139
with PostgreSQL and the Oracle
 database and an ASC/DESC index not
 pipelined, 140

H

hash join, 101
HASH GROUP BY, 169
HASH JOIN (Oracle), 169
HASH Join (PostgreSQL), 175
Hash Match, 183
Hash Match (Aggregate), 184
heap table, 3, 122
 creating in SQL Server, 127
Hibernate
 eager fetching, 96
 ILIKE uses LOWER, 98
 updates all columns, 164
hint, 19

I

IMMUTABLE (PostgreSQL), 30
INCLUDE, 121
index
 covering, 117
 fulltext, 48
 function-based, 24
 case insensitive, 24
 mathematical expressions, 77
 join, 50
 limits
 MySQL, Oracle, PostgreSQL, 121
 SQL Server, 122
 merge, 49
 multi-column, 12
 wrong order (effects), 81
 partial, 51
 prefix (MySQL), 121
 secondary, 123
index in MySQL execution plans, 189
index-only scan, 116
index-organized table, 122
 database support, 127
Index Cond (PostgreSQL), 177
INDEX FAST FULL SCAN, 168

INDEX FULL SCAN, 168
Index Only Scan (PostgreSQL), 174
INDEX RANGE SCAN, 167
Index Scan (PostgreSQL), 174
Index Seek, 182
INDEX UNIQUE SCAN, 167
 when accessing an IOT, 124
INTERNAL_FUNCTION, 67
IOT (index-organized table), 122

J

join, 91
 full outer, 109

K

Key Lookup (Clustered), 182

L

lazy fetching
 for scalar attributes (columns), 104
leaf node, 2
split, 160
LIKE, 45
 alternatives, 48
 as index filter predicate, 112
 on DATE column, 67
 on DATE columns, 67
limit (MySQL, PostgreSQL), 144
logarithmic scalability, 7
LOWER, 24

M

Merge Join, 109
 PostgreSQL, 175
 SQL Server, 183
MERGE JOIN (Oracle), 169
multi-block read
 for a full table scan, 13
 for a INDEX FAST FULL SCAN, 168
MVCC, 163
affects PostgreSQL index-only scan, 174
myths
 dynamic SQL is slow, 72, 74
 most selective column first
 disproof, 43
 origin, 49
 Oracle cannot index NULL, 56

N

N+1 problem, 92
Nested Loops, 92
 PostgreSQL, 175
 SQL Server, 183
NESTED LOOPS (Oracle), 168

SQL Optimizer

The query optimizer, or query planner, is the database component that transforms an SQL statement into an execution plan. This process is also called *compiling* or *parsing*. There are two distinct optimizer types.

- *Cost-based optimizers* (CBO) generate many execution plan variations and calculate a *cost* value for each plan. The cost calculation is based on the operations in use and the estimated row numbers. In the end the cost value serves as the benchmark for picking the “best” execution plan.
- *Rule-based optimizers* (RBO) generate the execution plan using a hardcoded rule set. Rule based optimizers are less flexible and are seldom used today.

The **execution plan is cached** so it can be reused when the same query arrives again before it is removed from the cache.

Statistics

A cost-based optimizer uses statistics about tables, columns, and indexes.

Most statistics are collected on the **column** level:

- the number of distinct values
- the smallest and largest values (data range)
- the number of NULL occurrences
- the column histogram (data distribution).

The most important statistical value for a **table** is its size (in rows and blocks).

The most important **index** statistics are

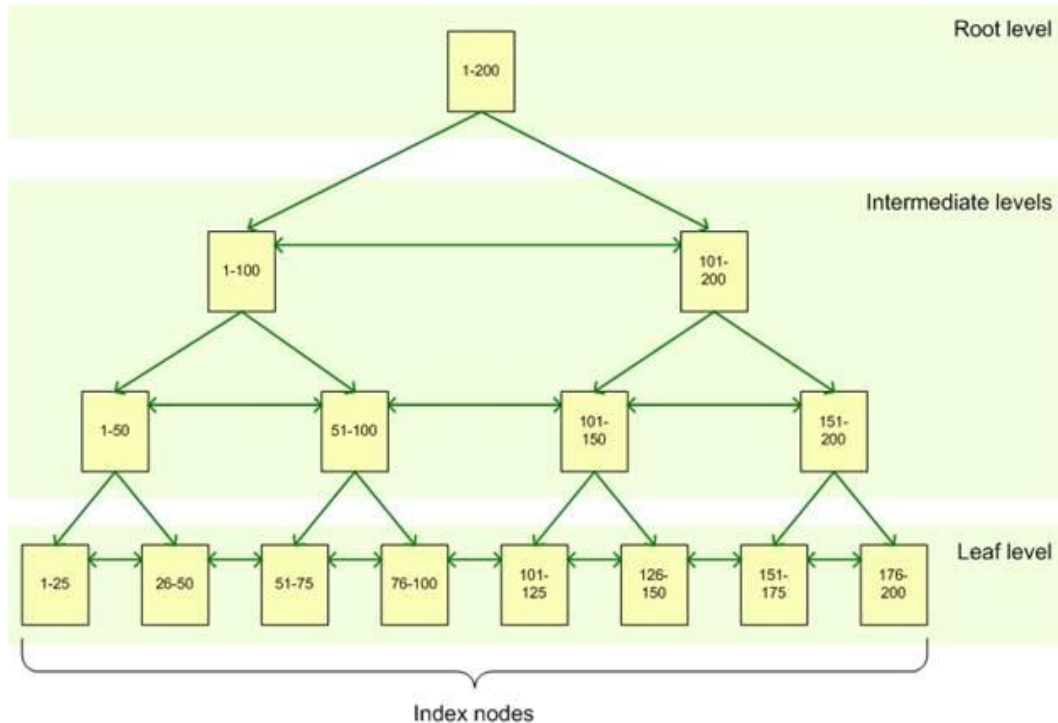
- the tree depth
- the number of leaf nodes
- the number of distinct keys

The optimizer uses these values to estimate the selectivity of the **where** clause predicates.

Impact of Cost-based optimizers

- Currently, all optimizers in major DBMS are cost-based.
= Execution plan is chosen based on **statistics**.
- As a human, we can only apply rules.
- But, DBMS might decide not to use an index, because “it’s not worth the effort”.
- E.g. where gender=‘F’ and ‘F’ applies to 50% of records
→ index on gender probably not used.

Indexes as B-trees

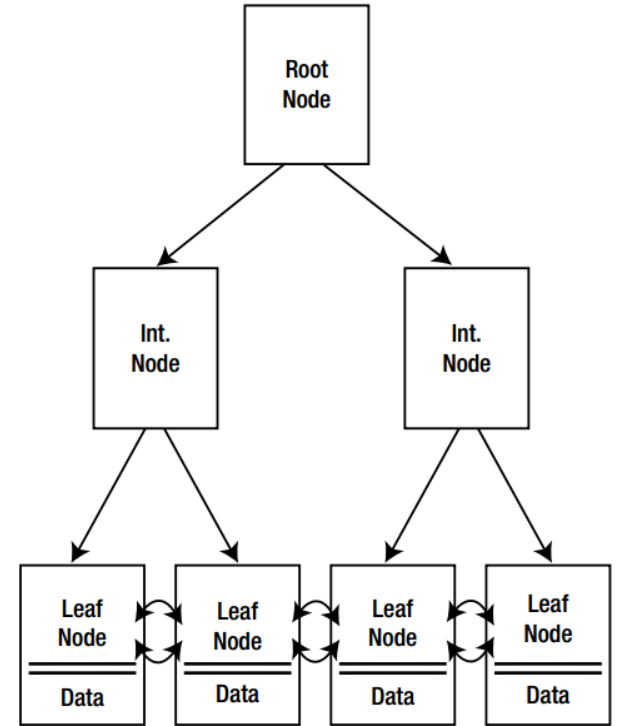


Clustered vs. Non-clustered indexes

- See the short version (2min)
https://www.youtube.com/watch?v=AINh6_LqnDM
- See the long version (6min)
<https://www.youtube.com/watch?v=ITcOiLSfVJQ>

Clustered index

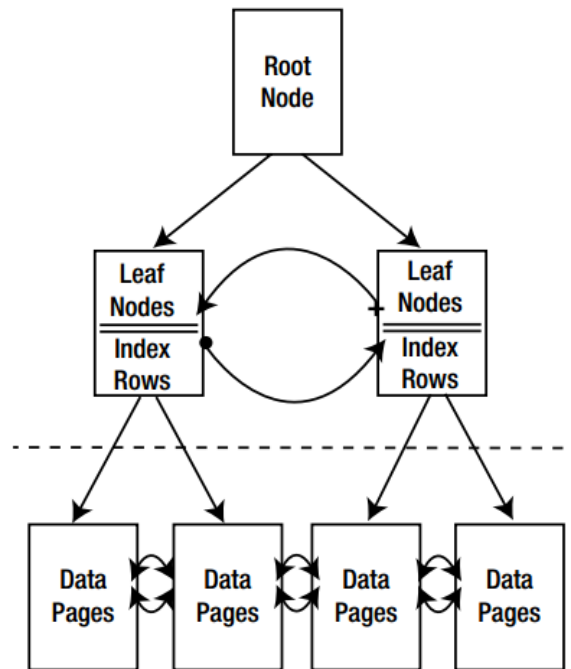
- The physical order of the rows in a table corresponds to the order in the clustered index.
- As a consequence, each table can have only one clustered index.
- The clustered index imposes unique values and the primary key constraint
- Advantages as opposed to table scan:
 - double linked list ensures order when reading sequential records
 - no forward pointers necessary



Int. Node = intermediate (tussenliggende) node

Non clustered index

- default index
- slower than clustered index
- > 1 per table allowed
- Forward and backward pointers between leaf nodes
- each *leaf* contains key value and *row locator*
 - to position in clustered index if it exists
 - otherwise to heap



Non clustered index

- If query needs more fields than present in index, these fields have to be fetched from data pages.
- When reading via non-clustered index:

either:

- RID lookup = bookmark lookups to the heap using RID's (= row identifiers)

or:

- key lookup = bookmark lookups to a clustered index, if present

Use of indexes with functions and wildcards

- In the case of '%r%' (with a leading wildcard) the index can't be used for searching
- However, it may be advantageous to include the corresponding field in a covering index.
- That way, the data that is needed is “clustered” (= stored together”), so fewer blocks have to be read.

Contents

- Introduction
- Clustered & Non-clustered Indexes
- **Covering Indexes**
- Concatenated Indexes
- Working with indexes
- Rules of thumb
- Quiz
- Materialized views
- Index statistics
- Storage & partitions

Covering index

- If a non clustered index not completely *covers* a query, SQL Server performs a lookup for each row to fetch the data
- Covering index = non-clustered index containing all columns necessary for a certain query
- With SQL Server you can add extra columns to the index (although those columns are not indexed!)

Covering index: example

(db xtreme, with script EmployeeIdx):

Current indexes on table Employee: each index indexes a single field.

- [-]  **dbo.Employee**
 - [+]  Columns
 - [+]  Keys
 - [+]  Constraints
 - [+]  Triggers
 - [-]  Indexes
 -  EmpBirthdate (Non-Unique, Non-Clustered)
 -  EmpFirstName (Non-Unique, Non-Clustered)
 -  EmpLastName (Non-Unique, Non-Clustered)
 -  EmpSalary (Non-Unique, Non-Clustered)
 -  PK_Employee (Clustered)

Covering index: example (db xtreme, with script EmployeeIdx):

SQLQuery1.sql - NB1...(EDU\jcor864 (56))*

```
1 select lastname from employee where lastname='Duffy';
2
3 select lastname, title from employee where lastname='Duffy';
4
```

121 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 33%

SELECT [lastname] FROM [employee] WHERE [lastname]=@1

Index Seek (NonClustered)
[Employee].[EmpLastName]
Cost: 100 %

- Index seek via nonclustered index for seeking lastname = 'Duffy'

Query 2: Query cost (relative to the batch): 67%

SELECT [lastname],[title] FROM [employee] WHERE [lastname]=@1

SELECT
Cost: 0 %

Nested Loops (Inner Join)
Cost: 0 %

Index Seek (NonClustered)
[Employee].[EmpLastName]
Cost: 50 %

Key Lookup (Clustered)
[Employee].[PK_Employee]
Cost: 50 %

- Key loop up in clustered index (= data) for fetching Title (not in index)

Covering index: example (cont'd)

Solution: covering index via INCLUDE

```
create nonclustered index EmpLastName_Incl_Title  
ON employee(lastname) INCLUDE (title);
```

SQLQuery1.sql - NB1...(EDU\jcor864 (56))

```
1 select lastname from employee where lastname='Duffy';  
2  
3 select lastname,title from employee where lastname='Duffy';
```

121 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 50%
SELECT [lastname] FROM [employee] WHERE [lastname]=@1

Index Seek (NonClustered)
[Employee].[EmpLastName_Incl_Title]
Cost: 100 %

SELECT
Cost: 0 %

Query 2: Query cost (relative to the batch): 50%
SELECT [lastname],[title] FROM [employee] WHERE [lastname]=@1

Index Seek (NonClustered)
[Employee].[EmpLastName_Incl_Title]
Cost: 100 %

SELECT
Cost: 0 %

Use of indexes with functions and wildcards

1 `select lastname,title`
2 `from Employee`
3 `where lastname = 'Preston'`
4
5 `select lastname,title`
6 `from Employee`
7 `where substring(lastname,1,1) = 'r'`
8
9 `select lastname,title`
10 `from Employee`
11 `where lastname like 'r%'`
12

231% No issues found

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 4%
`SELECT [lastname],[title] FROM [Employee] WHERE [lastname]='P'`

Index Seek (NonClustered)
[Employee].[EmpLastName_Icd_Title]
Cost: 100 %
0.000s
1 of 1
1 (100%)

Query 2: Query cost (relative to the batch): 84%
`select lastname,title from Employee where substring(lastname,1,1) = 'r'`

Index Scan (NonClustered)
[Employee].[EmpLastName_Icd_Title]
Cost: 97 %
0.004s
1963 of 1957 (100%)

Query 3: Query cost (relative to the batch): 12%
`select lastname,title from Employee where lastname like 'r%'`

Index Seek (NonClustered)
[Employee].[EmpLastName_Icd_Title]
Cost: 100 %
0.001s
1963 of 1960 (100%)

Contents

- Introduction
- Clustered & Non-clustered Indexes
- Covering Indexes
- **Concatenated Indexes**
- Working with indexes
- Rules of thumb
- Quiz
- Materialized views
- Index statistics
- Storage & partitions

1 index with several columns vs. several indexes with 1 column

```
create nonclustered index EmpLastName ON employee(lastname);  
+  
create nonclustered index EmpFirstname ON employee(firstname);
```

OR

?

```
create nonclustered index EmpLastNameFirstname ON  
employee(lastname, firstname);
```

1 index with several columns vs. several indexes with 1 column

Rule in SQL Server:

When querying (ex. in where-clause) only 2nd and or 3th, ... field of index, it is not used. This directly follows from the B-tree table structure of the composed index

So: `SELECT` LASTNAME, FIRSTNAME
 `FROM` EMPLOYEE2
 `WHERE` FIRSTNAME = 'Chris';

does **not use** the double index

Conclusion: make your indexes according to the most commonly used queries.

1 index with several columns or several indexes with 1 column?

Test:

only combined index on
Lastname, Firstname:

create index
EmpLastnameFirstname
on
Employee2
(lastname,firstname)

The screenshot displays two SQL queries and their corresponding execution plans in SQL Server Enterprise Manager.

Query 1: `select lastname,firstname from Employee2 where lastname = 'Preston'`

Query 2: `select lastname,firstname from Employee2 where firstname = 'Chris'`

Execution Plan for Query 1: The plan shows an "Index Seek (NonClustered)" operation on the index `[Employee2].[EmpLastnameFirstname]`. The cost is 100%, and it returns 1 of 1 rows (100%).

Execution Plan for Query 2: The plan shows an "Index Scan (NonClustered)" operation on the index `[Employee2].[EmpLastnameFirstname]`. The cost is 100%, and it returns 3 of 3 rows (100%).

1 index with several columns or several indexes with 1 column?

Extra index on Firstname:

```
create nonclustered index
EmpFirstname ON
employee2(firstname);
```

Not an optimal improvement
because of fetching lastname
through lookup

→ covering index with include
'lastname'

→ See further

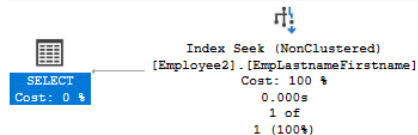
```
1  select lastname,firstname
2  from Employee2 where lastname = 'Preston'
3
4  select lastname,firstname
5  from Employee2 where firstname = 'Chris'
6
```

231 % No issues found

Results Messages Execution plan

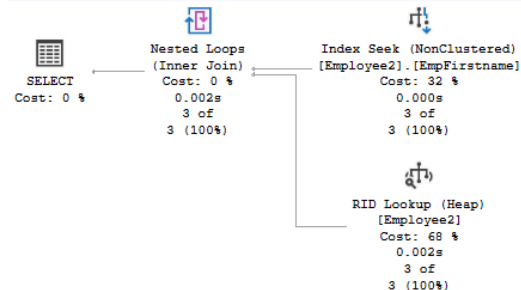
Query 1: Query cost (relative to the batch): 24%

```
SELECT [lastname],[firstname] FROM [Employee2] WHERE [lastname]=@1
```



Query 2: Query cost (relative to the batch): 76%

```
SELECT [lastname],[firstname] FROM [Employee2] WHERE [firstname]=@1
```



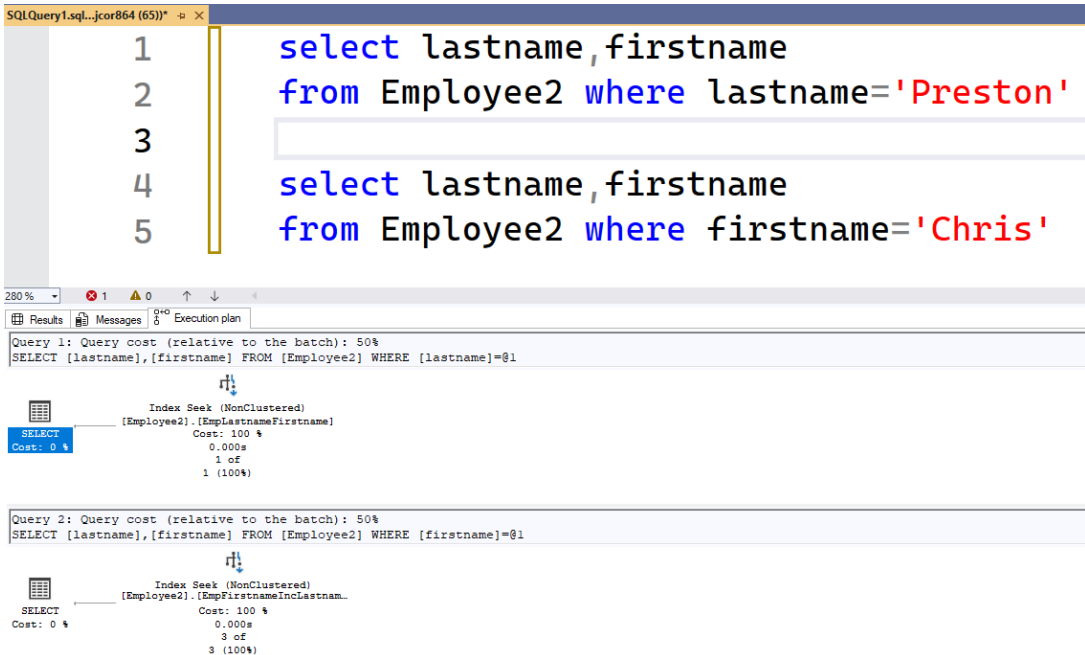
1 index wit several columns or several indexes with 1 column

With extra index on Firstname and **covering** of Lastname

```
create nonclustered index EmpFirstnameIncLastname ON employee2(firstname)  
INCLUDE (lastname);
```

```
1 select lastname,firstname  
2 from Employee2 where lastname='Preston'  
3  
4 select lastname,firstname  
5 from Employee2 where firstname='Chris'
```

now query execution
times are equal.



Sort order with concatenated indexes

```
CREATE NONCLUSTERED INDEX  
EmpLastnameTitle ON Employee2  
(  
    LastName ASC,  
    Title ASC  
)
```

Index can be used in reverse order,
but you can't mix the order of the
two fields.

1 select lastname,title
2 from Employee2
3 order by lastname asc, title asc;
4
5 select lastname,title
6 from Employee2
7 order by lastname desc, title desc;
8
9 select lastname,title
10 from Employee2
11 order by lastname asc, title desc;

280 % No issues found

Results Messages 8th Execution plan

Query 1: Query cost (relative to the batch): 5%
select lastname,title from Employee2 order by lastname asc, title asc

Index Scan (NonClustered)
[Employee2].[EmpLastnameTitle]
Cost: 100 %
0.006s
18790 of
18790 (100%)

Query 2: Query cost (relative to the batch): 5%
select lastname,title from Employee2 order by lastname desc, title desc

Index Scan (NonClustered)
[Employee2].[EmpLastnameTitle]
Cost: 100 %
0.010s
18790 of
18790 (100%)

Query 3: Query cost (relative to the batch): 90%
select lastname,title from Employee2 order by lastname asc, title desc

Sort
Cost: 94 %
0.017s
18790 of
18790 (100%)

Index Scan (NonClustered)
[Employee2].[EmpLastnameTitle]
Cost: 6 %
0.006s
18790 of
18790 (100%)

Contents

- Introduction
- Clustered & Non-clustered Indexes
- Covering Indexes
- Concatenated Indexes
- **Working with indexes**
- Rules of thumb
- Quiz
- Materialized views
- Index statistics
- Storage & partitions

Creation of indexes: syntax

```
CREATE [ UNIQUE ] [ CLUSTERED | NONCLUSTERED ] INDEX index_name  
ON <object> ( column [ ASC | DESC ] [ ,...n ] )  
[ INCLUDE ( column_name [ ,...n ] ) ]
```

- **unique** all values in the indexed column should be unique
- remark:
 - when defining an index the table can be empty or filled.
 - columns in a **unique** index should have the **not null** constraint

Removing indexes

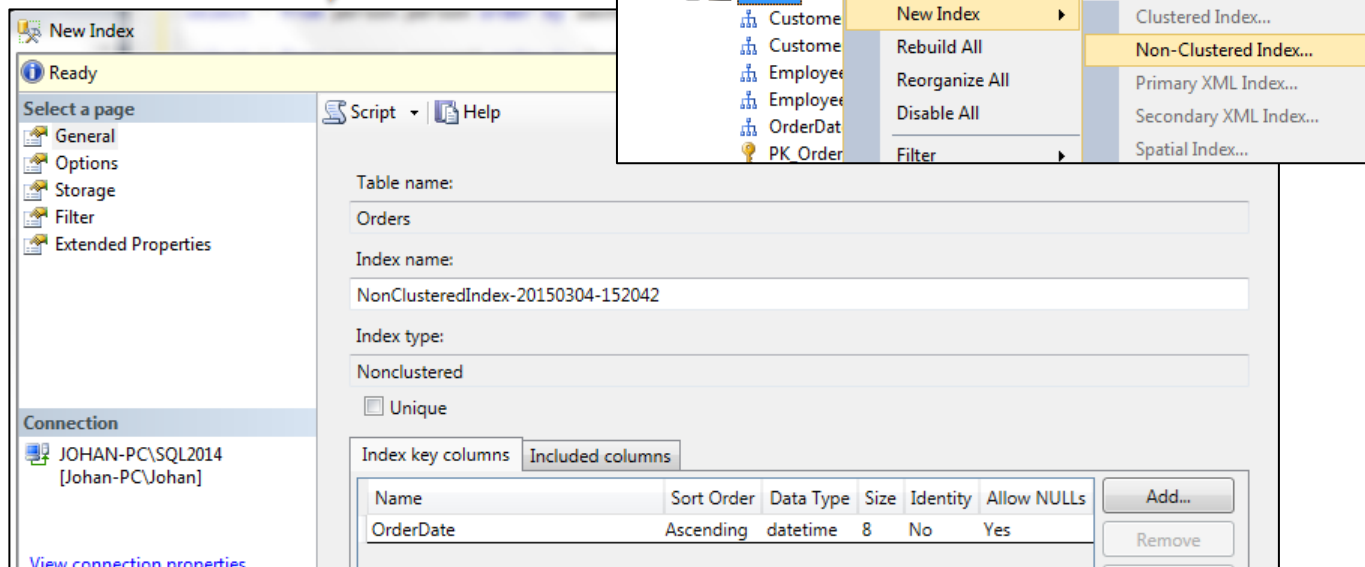
```
DROP INDEX table_name.index [,...n]
```

deleting index

Working with indexes

SQL Server

Management Studio



When to use an index

- **which columns should be indexed?**
 - primary and unique columns are indexed automatically
 - foreign keys often used in joins
 - columns often used in search conditions (WHERE, HAVING, GROUP BY) or in joins
 - columns often used in the ORDER BY clause
- **which columns should not be indexed?**
 - columns that are rarely used in queries
 - columns with a small number of possible values (e.g. gender)
 - columns in small tables
 - columns of type bit, text or image

Contents

- Introduction
- Clustered & Non-clustered Indexes
- Covering Indexes
- Concatenated Indexes
- Working with indexes
- **Rules of thumb**
- Quiz
- Materialized views
- Index statistics
- Storage & partitions

Rules of thumb

DB xtreme:

```
CREATE INDEX EmpFirstName ON  
Employee (FirstName ASC);
```

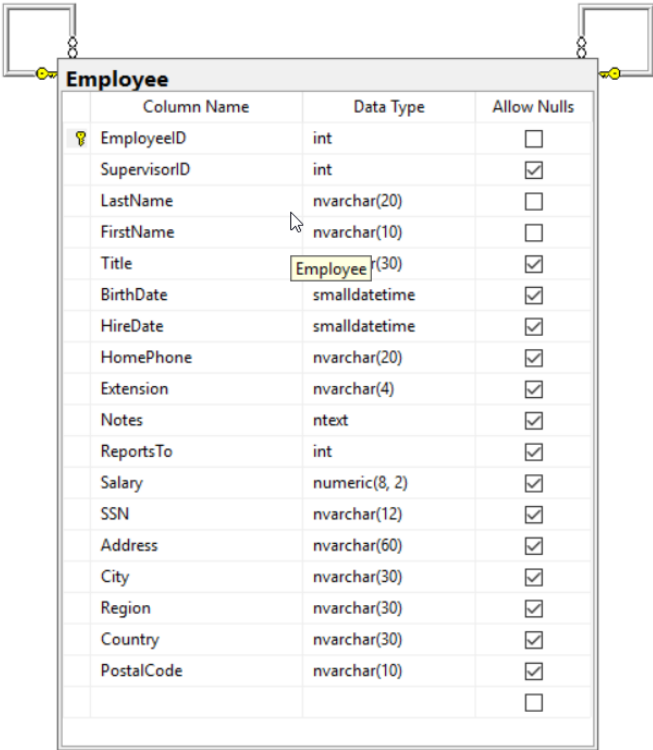
```
CREATE INDEX EmpLastName ON Employee  
(LastName ASC);
```

```
CREATE INDEX EmpDOB ON Employee  
(BirthDate ASC);
```

```
CREATE INDEX EmpSalary ON Employee  
(Salary ASC);
```

The following slides provides some general rules of thumb that are applicable in most cases on most databases. They are not carved in stone.

The employee table used in the examples has about 20.000 records.



The image shows a screenshot of a database table structure for a table named 'Employee'. The table has 19 columns. The first column, 'EmployeeID', is marked as the primary key with a yellow key icon. The 'Allow Nulls' column shows checkboxes for each column, with most set to 'No' (unchecked) and some set to 'Yes' (checked). The 'Title' column has a value 'Employee' in parentheses next to its data type 'nvarchar(30)'. The table is displayed in a window with a title bar and standard window controls.

Column Name	Data Type	Allow Nulls
EmployeeID	int	☐
SupervisorID	int	☒
LastName	nvarchar(20)	☐
FirstName	nvarchar(10)	☐
Title	Employee nvarchar(30)	☒
BirthDate	smalldatetime	☒
HireDate	smalldatetime	☒
HomePhone	nvarchar(20)	☒
Extension	nvarchar(4)	☒
Notes	ntext	☒
ReportsTo	int	☒
Salary	numeric(8, 2)	☒
SSN	nvarchar(12)	☒
Address	nvarchar(60)	☒
City	nvarchar(30)	☒
Region	nvarchar(30)	☒
Country	nvarchar(30)	☒
PostalCode	nvarchar(10)	☒
		☐

(1) avoid the use of functions

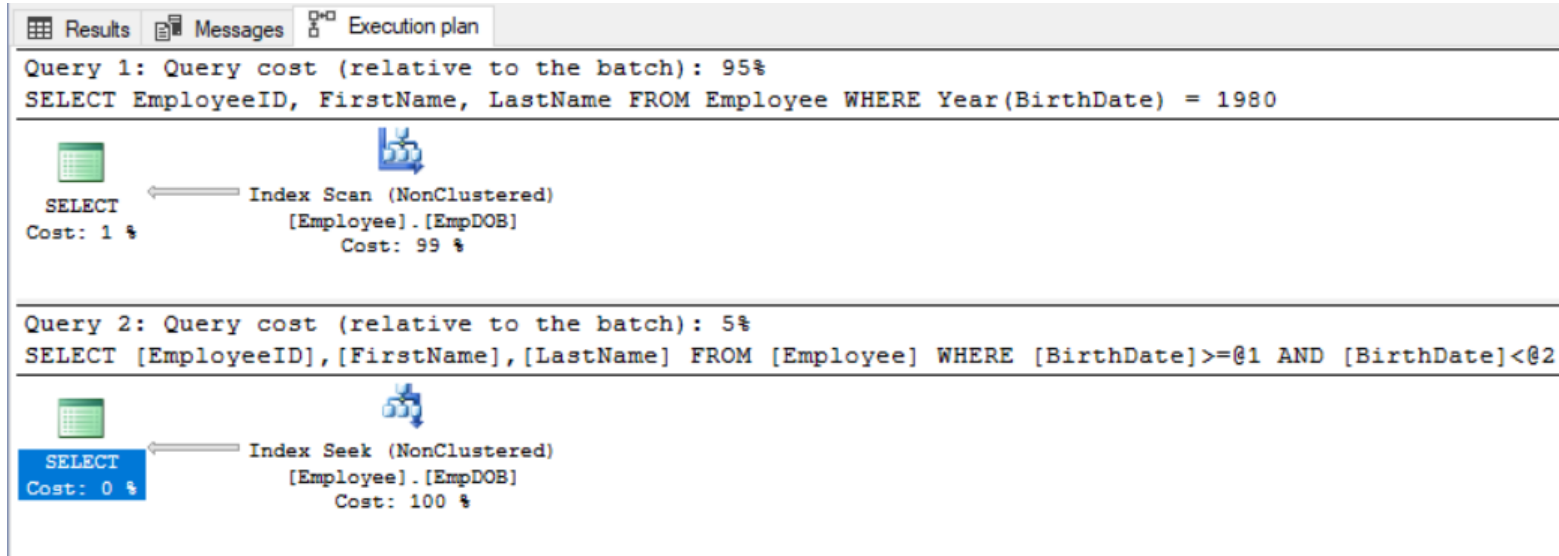
-- BAD

```
SELECT EmployeeID, Birthdate
FROM Employee
WHERE Year(BirthDate) = 1980;
```

-- GOOD

```
SELECT EmployeeID, Birthdate
FROM Employee
WHERE BirthDate >= '1980-01-01'
AND BirthDate < '1981-01-01';
```

(1) avoid the use of functions



- Index Scan: index is used but it is scanned from the start till the searched records are found.
- Index Seek: tree structure of index is used, resulting in very fast data retrieval.

(1) avoid the use of functions

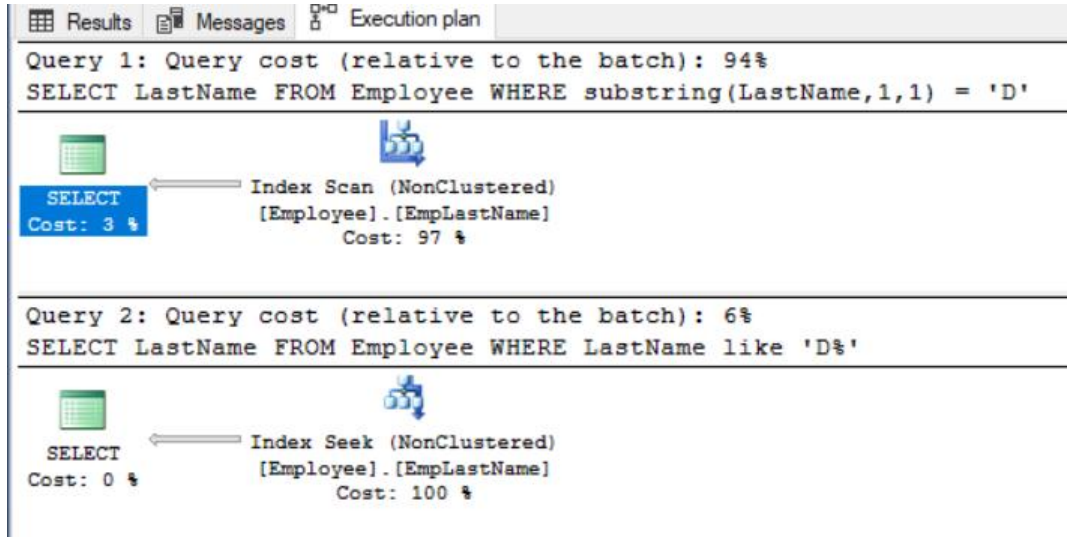
-- BAD

```
SELECT LastName  
FROM Employee  
WHERE substring(LastName,1,1) = 'D';
```

-- GOOD

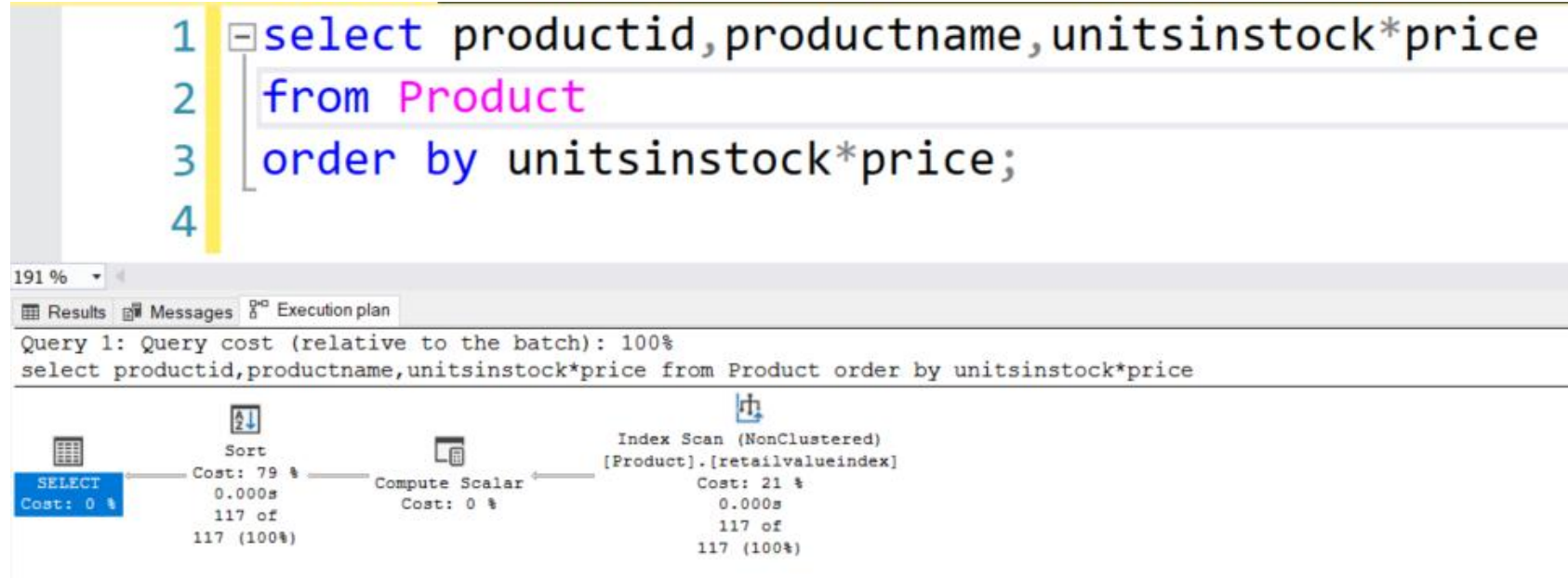
```
SELECT LastName  
FROM Employee  
WHERE LastName like 'D%';
```

(1) avoid the use of functions



- Index Scan: index is used but it is scanned from the start till the searched records are found.
- Index Seek: tree structure of index is used, resulting in very fast data retrieval.

(1) avoid the use of functions: calculations



- Expensive sort is done in query
- Can' be avoided by using index because of formula

(1) avoid the use of functions : calculations

- Some DBMS's (like Oracle and PostgreSQL) support the creation of function based indexes
- SQL Server doesn't but it knows the concepts of computed columns and materialized views.
- By creating an index on a **computed column** or by creating a **materialized view** (see further) you can simulate the effect of a function based index.
- Computed column:

```
ALTER TABLE dbo.Product ADD RetailValue  
AS (Unitsinstock * Price * 1.5);
```

```
create index retailvalueindex on product(retailvalue)  
include(productname,unitsinstock,price);
```

```
select productid,productname,unitsinstock,price,retailvalue  
from Product order by RetailValue;
```

(1) avoid the use of functions : calculations

```
5 ALTER TABLE dbo.Product ADD RetailValue AS (Unitsinstock * Price * 1.5);
6
7 create index retailvalueindex on product(retailvalue) include(productname);
8
9 select productid,productname,retailvalue
10 from Product order by RetailValue;
11
```

119 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

select productid,productname,retailvalue from Product order by RetailValue

```
graph RL
    subgraph Plan
        direction LR
        S[SELECT  
Cost: 0 %] <--> CS[Compute Scalar  
Cost: 0 %]
        CS <--> IS[Index Scan (NonClustered)  
[Product].[retailvalueindex]  
Cost: 100 %  
0.000s  
117 of  
117 (100%)]
    end
```

- No more sort in query
- Sort is now through index, which is always sorted.

(2) avoid calculations, isolate columns

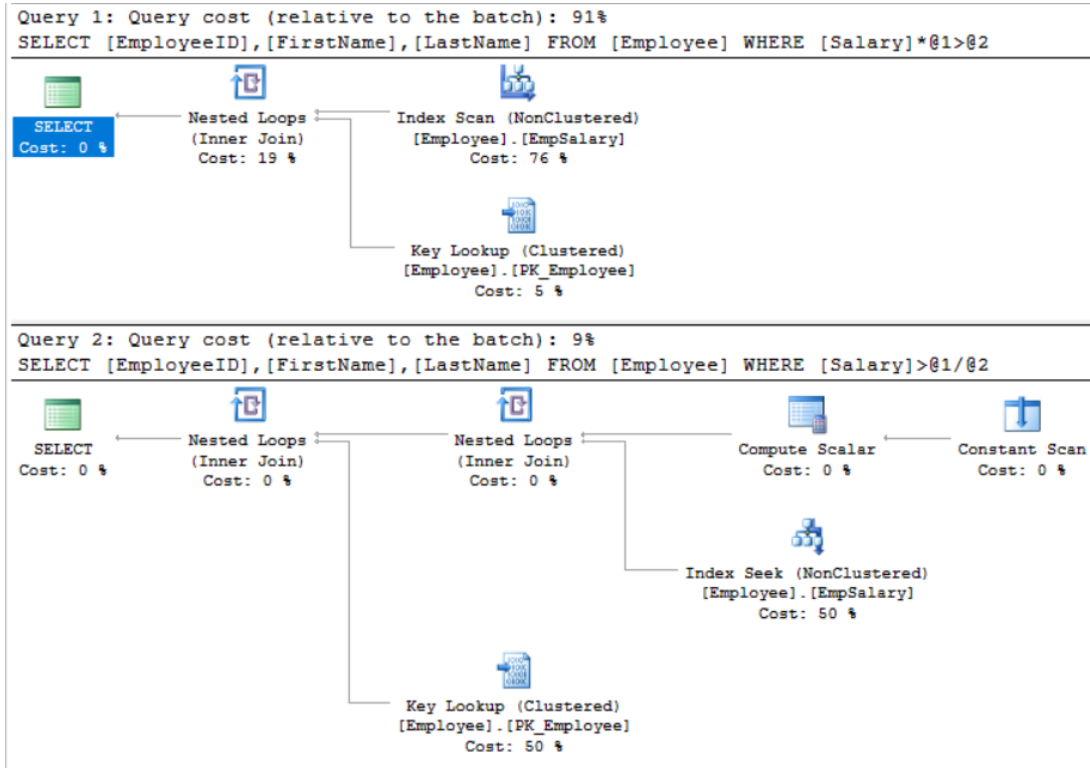
-- BAD

```
SELECT EmployeeID, FirstName, LastName  
FROM Employee  
WHERE Salary*1.10 > 100000;
```

-- GOOD

```
SELECT EmployeeID, FirstName, LastName  
FROM Employee  
WHERE Salary > 100000/1.10;
```

(2) avoid calculations, isolate columns



Key lookup:

The non-clustered index EmpSalary, holds in each leaf a reference to the location of the total record in the clustered index. Following this reference is called “key lookup”.

(3) prefer OUTER JOIN above UNION

-- BAD

```
SELECT lastname, firstname, orderid
from Employee e join Orders o on e.EmployeeID = o.employeeid
union
select lastname, firstname, null
from Employee
where EmployeeID not in (select EmployeeID from Orders)
```

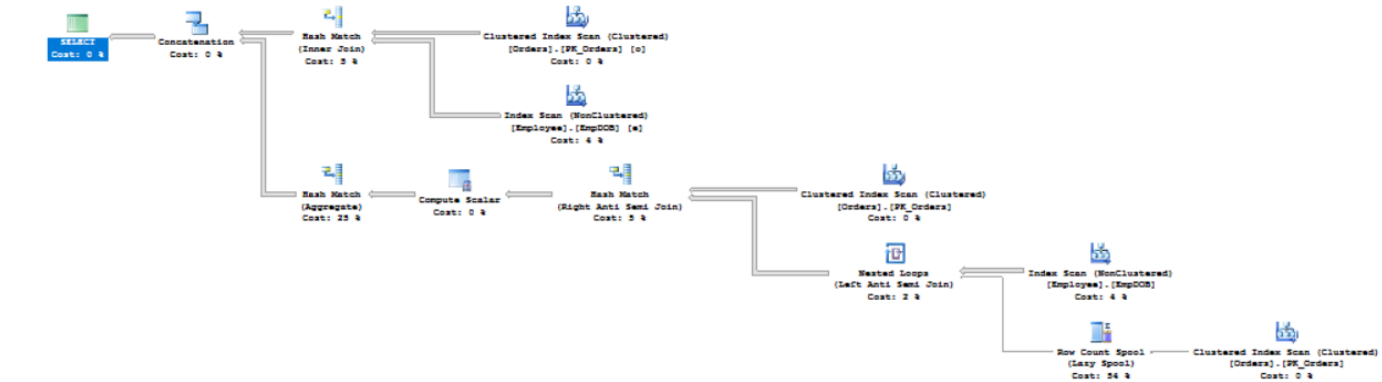
-- GOOD

```
SELECT lastname, firstname, orderid
from Employee e left join Orders o on e.EmployeeID = o.employeeid;
```

(3) prefer OUTER JOIN above UNION

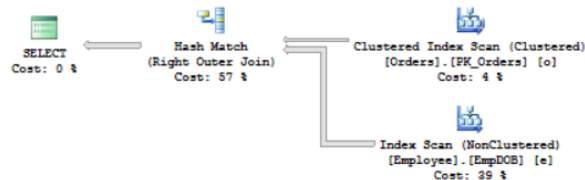
Query 1: Query cost (relative to the batch): 91%

```
SELECT lastname,firstname,orderid from Employee e join Orders o on e.EmployeeID = o.employeeid union select lastname,firstname,null fr
```



Query 2: Query cost (relative to the batch): 9%

```
SELECT lastname,firstname,orderid from Employee e left join Orders o on e.EmployeeID = o.employeeid
```



(4) avoid ANY and ALL

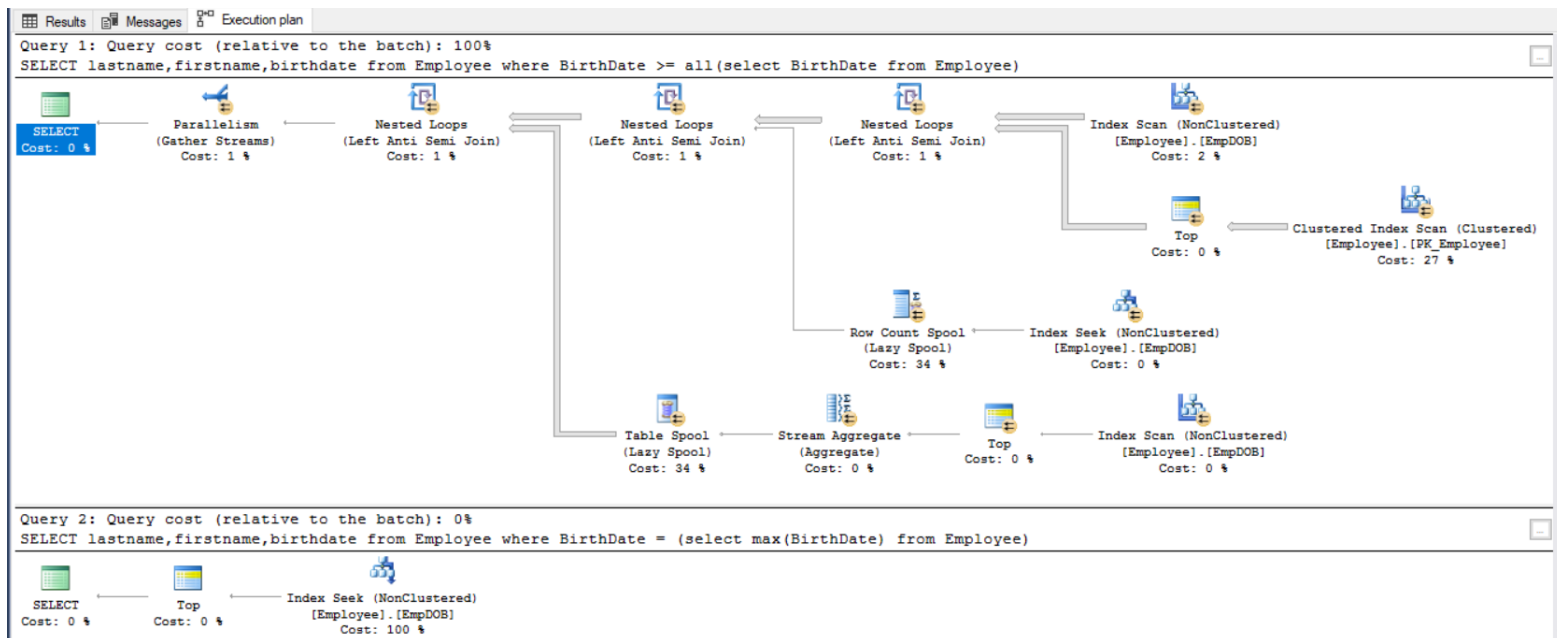
-- BAD

```
SELECT lastname, firstname, birthdate
from Employee
where BirthDate >= all(select BirthDate from Employee)
```

-- GOOD

```
SELECT lastname, firstname, birthdate
from Employee
where BirthDate = (select max(BirthDate) from Employee)
```

(4) avoid ANY and ALL



(5) Index for equality first — then for ranges.

```
select lastname,birthdate,country  
from Employee  
where BirthDate >= '1980-01-01' and BirthDate <= '1990-12-31'  
and Country = 'Canada'
```

```
create index IdxCountryBirthdate on Employee(country,birthdate)
```

(6) Check the SQL code that is generated by your ORM tool or framework

- get to know how your ORM tool generates SQL queries.
- See e.g. `print(query)` when using SQLAlchemyORM in Python
- e.g. Sometimes ORM tools use UPPER and LOWER without the developer's knowledge. Hibernate, for example, injects an implicit LOWER for case-insensitive searches.

(7) Avoid dynamic SQL whenever possible

Dynamic SQL always involves late binding

- Late binding performs the binding of SQL-statements at runtime
 - additional flexibility is offered (“dynamic SQL”)
 - syntax errors or authorisation issues will remain hidden until the program is executed
 - testing the application can be harder (use “PRINT”)
 - less efficient for queries that must be executed multiple times
 - Risk of SQL injection attacks
- Not allowed in User Defined Functions in MS SQL Server (= risk of side effects)

(7) Avoid dynamic SQL whenever possible

```
-- Dynamic SQL example in t-SQL:  
declare @region varchar(10);  
set @region = 'OR';  
declare @sqlstring varchar(100) = 'select * from supplier  
where region=''' + @region + ''';  
exec (@sqlstring);
```

(8) Use bind variables

- Bind parameters — also called dynamic parameters or bind variables — are an alternative way to pass data to the database.
- Instead of putting the values directly into the SQL statement, you just use a placeholder like `?`, `:name` or `@name` and provide the actual values using a separate API call.
- Databases with an execution plan cache like SQL Server can reuse an execution plan when executing the same statement multiple times. It saves effort in rebuilding the execution plan but works only if the SQL statement is exactly the same.

(8) Use bind variables

C#

Without bind parameters:

```
int subsidiary_id;  
SqlCommand cmd = new SqlCommand(  
    "select first_name, last_name"  
    + " from employees"  
    + " where subsidiary_id = " + subsidiary_id  
    , connection);
```

Using a bind parameter:

```
int subsidiary_id;  
SqlCommand cmd =  
    new SqlCommand(  
        "select first_name, last_name"  
        + " from employees"  
        + " where subsidiary_id = @subsidiary_id"  
        , connection);  
cmd.Parameters.AddWithValue("@subsidiary_id", subsidiary_id);
```

(8) Use bind variables

JAVA

Without bind parameters:

```
int subsidiary_id;  
Statement command = connection.createStatement(  
    "select first_name, last_name"  
    + " from employees"  
    + " where subsidiary_id = " + subsidiary_id  
);
```

Using a bind parameter:

```
int subsidiary_id;  
PreparedStatement command = connection.prepareStatement(  
    "select first_name, last_name"  
    + " from employees"  
    + " where subsidiary_id = ?"  
);  
command.setInt(1, subsidiary_id);
```

(8) Use bind variables

Cursor Sharing and Auto Parameterization

- The more complex the optimizer and the SQL query become, the more important execution plan caching becomes.
- The SQL Server and Oracle databases have features to automatically replace the literal values in a SQL string with bind parameters.
- These features are called CURSOR_SHARING (Oracle) or forced parameterization (SQL Server)
- These are workarounds for applications that do not use bind parameters at all.
- Enabling these features prevents developers from intentionally using literal values.

(8) Use bind variables

- Default in SQL Server: simple parameterization
→ optimizer will choose to use parameterization or not.
- Check by:
`SELECT name, is_parameterization_forced FROM sys.databases`
- Can be turned into forced parameterization
- See
 - <https://www.mssqltips.com/sqlservertip/2935/sql-server-simple-and-forced-parameterization/>
 - <https://docs.microsoft.com/en-us/sql/relational-databases/performance/specify-query-parameterization-behavior-by-using-plan-guides?view=sql-server-ver15>

(9) Execute joins in the database.

- Don't implement in your application what the database can do better
- Database is optimized for efficient data retrieval
- Limit network traffic

(10) Avoid unnecessary joins.

- Reading from many scattered tables is sensitive to disk seek latencies.
- JOIN can process only two tables at a time

Contents

- Introduction
- Clustered & Non-clustered Indexes
- Covering Indexes
- Concatenated Indexes
- Working with indexes
- Rules of thumb
- **Quiz**
- Materialized views
- Index statistics
- Storage & partitions

Quiz 1/5

Is the following index a good fit for the query?

```
CREATE INDEX tbl_idx ON tbl (date_column);
```

```
SELECT * FROM tbl  
WHERE YEAR(date_column) = 2017;
```

- A. Good fit: No need to change anything
- B. Bad fit: Changing the index or query could improve performance

Quiz 2/5

Is the following index a good fit for the query?

```
CREATE INDEX tbl_idx ON tbl (a, date_column);
```

```
SELECT TOP 1 a,date_column FROM tbl  
  WHERE a = 12  
  ORDER BY date_column DESC;
```

- A. Good fit: No need to change anything
- B. Bad fit: Changing the index or query could improve performance

Quiz 3/5

Is the following index a good fit for both queries?

```
CREATE INDEX tbl_idx ON tbl (a, b);
```

```
SELECT a,b FROM tbl  
WHERE a = 123 AND b = 1;
```

```
SELECT a,b FROM tbl WHERE b = 123;
```

- A. Good fit: No need to change anything
- B. Bad fit: Changing the index or query could improve performance

Quiz 4/5

Is the following index a good fit for the query
(you may assume field text is of datatype varchar(50) ?

```
CREATE INDEX tbl_idx ON tbl (text);
```

```
SELECT text FROM tbl  
WHERE text LIKE 'TJ%';
```

- A. Good fit: No need to change anything
- B. Bad fit: Changing the index or query could improve performance

Quiz 5/5

This question is different.

First consider the following index and query:

```
CREATE INDEX tbl_idx ON tbl (a, date_column);
```

```
SELECT date_column, count(*) FROM tbl  
WHERE a = 123  
GROUP BY date_column;
```

How will the change affect performance:

- A. Same: Query performance stays about the same
- B. Not enough information: Definite answer cannot be given
- C. Slower: Query takes more time
- D. Faster: Query takes less time

Let's say this query returns at least a few rows.

To implement a new functional requirement, another condition ($b = 1$) is added to the where clause:

```
SELECT date_column, count(*)  
FROM tbl  
WHERE a = 123 AND b = 1  
GROUP BY date_column;
```

Contents

- Introduction
- Clustered & Non-clustered Indexes
- Covering Indexes
- Concatenated Indexes
- Working with indexes
- Rules of thumb
- Quiz
- **Materialized views**
- Index statistics
- Storage & partitions

Indexed views (or materialized views)

“NORMAL” view:

- Database view in SQL Server: SELECT statement is stored, no data stored.
- A view is NOT a snapshot of the data at the moment the view is created.
- Created using the CREATE VIEW statement.
- You can write queries against a view as if it were a table.
- When a view gets queried, the optimizer receives the full definition of the SELECT statement and uses that as the basis for optimizing the query against the view.
- Through the optimization process, some or all of the definition of the SELECT statement may be used to satisfy the query against the view. What degree of simplification occurs here is determined by a combination of the SELECT statement itself and the query against that SELECT statement.

Indexed views (or materialized views)

- A database view can be **materialized** on the disk by **creating a unique clustered index** on the view.
- Such a view is referred to as an *indexed view* or a *materialized view*.
- After a unique clustered index is created on the view, the view's result set is materialized immediately and persisted in physical storage in the database, saving the overhead of performing costly operations during query execution.
- This acts just like a regular clustered index: the clustered index IS the table storage.
- After the view is materialized, multiple nonclustered indexes can be created on the indexed view to speed up specific queries.
- This turns a view (again, just a query) into a real table with defined storage.

Indexed views: benefits

- Aggregations can be precomputed and stored in the indexed view to minimize expensive computations during query execution.
- Tables can be prejoined, and the resulting data set can be materialized.
- Combinations of joins or aggregations can be materialized.

Indexed views: restrictions

- The first index on the view must be a unique clustered index. Nonclustered indexes on an indexed view can be created only after the unique clustered index is created.
- The view definition must be *deterministic*—that is, it is able to return only one possible result for a given query. Functions like GETDATE(), RAND(), ... are non-deterministic and cannot be used.
- The indexed view must reference only base tables in the same database, not other views.
- The indexed view may contain float columns. However, such columns cannot be included in the clustered index key.
- The indexed view must be schema bound (see further) to the tables referred to in the view to prevent modifications of the table schema.
- There are several restrictions on the syntax of the view definition. (A list of the syntax limitations on the view definition is provided in SQL Server Books Online.). Examples:
 - OUTER JOINs, DISTINCT and UNION are not allowed
 - COUNT has to be replaced by COUNT_BIG
 - Only COUNT_BIG(*) is allowed, COUNT_BIG(<field name>) is not allowed

Indexed views: schema binding

- In SQL Server, a **schema** is a **logical collection of database objects such as tables, views, stored procedures, indexes, triggers, functions**.
- It can be thought of as a **container**, created by a database user. The database user who creates a schema is the schema owner. Default schema = dbo, but many schemas can be created in a single database.
- The indexed view must be *schema bound* to the tables referred to in the view to prevent modifications of the table schema (frequently a major problem).
- In SQL Server, views are not bound to the schema of the base tables by default. In such case we may change the schema of the base table at any time, regardless of the fact that the associated view may or may not work with the new schema. We can even drop an underlying table while keeping the associated view without any warning. In this case when the view is used, we will get an invalid object name error for the base table.
- If you want to create an index on a view or you want to preserve the base table schema once a view has been defined, in both cases you have to use the "WITH SCHEMABINDING" clause to bind the view to the schema of the base tables (db xtreme):

```
CREATE VIEW dbo.V_ProductsCustomer(productcode, customername, sumquantity)
WITH SCHEMABINDING
AS SELECT productid, customername, sum(quantity)
FROM dbo.customer
JOIN dbo.orders ON orders.customerid = customer.customerid
JOIN dbo.ordersdetail ON orders.orderid = ordersdetail.orderid
GROUP BY productid,customername
```

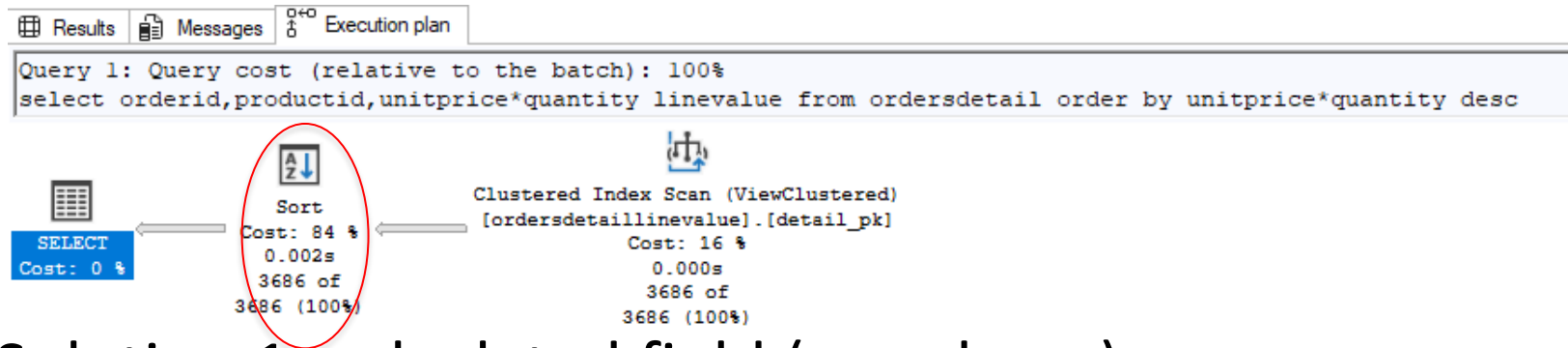
Indexed views: schema binding

- Be aware that if you do not use the schema name, **dbo** in this case, then you will get the following error while creating the view. *"Cannot schema bind view 'dbo.vw_sampleView' because name 'SAMPLETABLE' is invalid for schema binding. Names must be in two-part format and an object cannot reference itself."*
- So the steps are:
 1. Create view with schema binding (use schema name).
 2. Create unique clustered index on view to materialize the view (physical storage = index)
 3. Create any non clustered index on view to speed up queries.

Indexed view: example

- Query to optimize (db xtreme)

```
select orderid,productid,unitprice*quantity linevalue  
from ordersdetail  
order by unitprice*quantity desc;
```



- Solution 1: calculated field (see above)
- Solution 2: indexed view

Indexed view: example (cont'd)

Create view

```
create view dbo.ordersdetaillinevalue(orderid,productid,linevalue)
with schemabinding
as select orderid,productid,unitprice*quantity
from dbo.ordersdetail
```

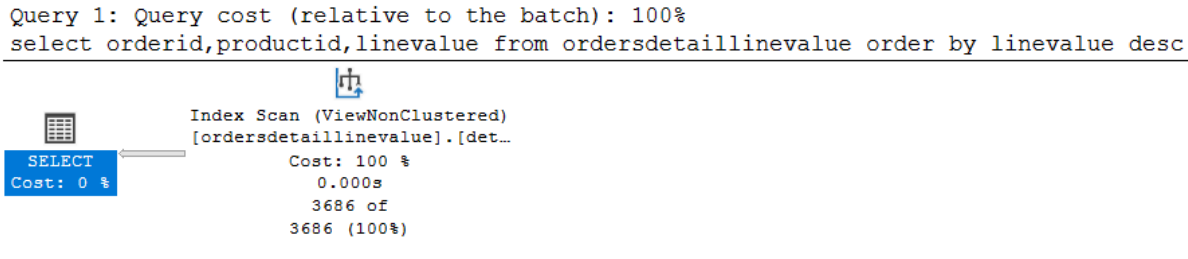
Create clustered index to create storage:

```
create unique clustered index detail_pk on ordersdetaillinevalue(orderid,productid)
```

Create index to speed up query:

```
create nonclustered index detail_idx on ordersdetaillinevalue(linevalue)
include(orderid,productid)
```

Check execution plan:



Contents

- Introduction
- Clustered & Non-clustered Indexes
- Covering Indexes
- Concatenated Indexes
- Working with indexes
- Rules of thumb
- Quiz
- Materialized views
- **Index statistics**
- Storage & partitions

Index usage statistics

Overview of indexes and their usage:

```
SELECT OBJECT_NAME(IX.OBJECT_ID) Table_Name
, IX.name AS Index_Name
, IX.type_desc Index_Type
, SUM(PS.[used_page_count]) * 8 IndexSizeKB
, IXUS.user_seeks AS NumOfSeeks
, IXUS.user_scans AS NumOfScans
, IXUS.user_lookups AS NumOfLookups
, IXUS.user_updates AS NumOfUpdates
, IXUS.last_user_seek AS LastSeek
, IXUS.last_user_scan AS LastScan
, IXUS.last_user_lookup AS LastLookup
, IXUS.last_user_update AS LastUpdate
FROM sys.indexes IX
INNER JOIN sys.dm_db_index_usage_stats IXUS ON IXUS.index_id = IX.index_id AND IXUS.OBJECT_ID = IX.OBJECT_ID
INNER JOIN sys.dm_db_partition_stats PS on PS.object_id=IX.object_id
WHERE OBJECTPROPERTY(IX.OBJECT_ID, 'IsUserTable') = 1
GROUP BY OBJECT_NAME(IX.OBJECT_ID) , IX.name , IX.type_desc , IXUS.user_seeks , IXUS.user_scans
, IXUS.user_lookups, IXUS.user_updates , IXUS.last_user_seek , IXUS.last_user_scan , IXUS.last_user_lookup
, IXUS.last_user_update
```

Index usage statistics

Table_Name	Index_Name	Index_Type	IndexSizeKB	NumOfSeeks	NumOfScans	NumOfLookups	NumOfUpdates	LastSeek	LastScan	LastLookup	LastUpdate
categorie	PK_Categorieën	CLUSTERED	16	1194	0	0	0	2020-10-22 00:01:55.287	NULL	NULL	NULL
factuur	PK_factuur	CLUSTERED	40	2	103	0	2	2020-10-15 17:45:00.670	2020-10-21 22:21:39.750	NULL	2020-10-15 17:45:00.670
klant	PK_klant	CLUSTERED	16	1612790	972	0	0	2020-10-22 00:01:55.283	2020-10-22 00:01:55.087	NULL	NULL
kost	PK_Bedrijfskosten	CLUSTERED	48	0	1592	0	0	NULL	2020-10-22 00:01:55.287	NULL	NULL
param	NULL	HEAP	16	0	44	0	16	NULL	2020-10-21 18:06:18.603	NULL	2020-10-21 18:06:18.603
project	IX_project	NONCLUSTERED	112	47	391	0	2	2020-10-20 10:53:07.650	2020-10-22 00:00:27.583	NULL	2020-10-16 15:10:45.563
project	NonClusteredIndex-20160428-231519	NONCLUSTERED	112	2	21	0	2	2020-10-21 18:02:57.003	2020-10-21 18:02:06.220	NULL	2020-10-16 15:10:45.563
project	PK_project	CLUSTERED	112	73	24447	2	4	2020-10-21 18:03:31.510	2020-10-22 00:01:55.287	2020-10-21 18:02:57.003	2020-10-16 15:10:45.563
timesheet	PK_timesheet_1	CLUSTERED	224	0	8	0	8	NULL	2020-10-21 18:30:01.473	NULL	2020-10-21 18:30:01.473
timesheet	PK_timesheet_1	CLUSTERED	224	45	45468	0	106	2020-10-20 10:53:07.650	2020-10-22 00:01:55.287	NULL	2020-10-20 10:53:07.650
timesheetbackup	NULL	HEAP	1496	0	0	0	16	NULL	NULL	NULL	2020-10-20 10:53:07.640

- Which tables need indexes, which indexes are seldom used?
- Statistics since last start of SQL Server service:
 - Seek: index seek
 - Scan: index scan
 - Update: updates of index
 - Lookup: key lookup from nonclustered index in clustered index

Contents

- Introduction
- Clustered & Non-clustered Indexes
- Covering Indexes
- Concatenated Indexes
- Working with indexes
- Rules of thumb
- Quiz
- Materialized views
- Index statistics
- **Storage & partitions**

Storage: files & filegroups

- Files & Filegroups
 - each DB consists of at least two files
 - Data file (.mdf)
 - log file (.ldf)
 - Optionally extra data files can be added (.ndf)
 - filegroup
 - Logic grouping of files for administration and allocation purposes
 - default: 1 filegroup per db-file
- Performance gain by organizing file groups
 - Store file groups on separate physical disks
 - Store data files and non clustered indexes on separate disks (clustered index IS data file)
 - Store log files on another disk than data and non clustered indexes for data security

Storage: files & filegroups

- Moving a table to another file group
 - Drop current clustered index. The table is then recreated on the heap.
 - Remember: clustered index = data file
 - Create new clustered index and specify filegroup with `CREATE CLUSTERED INDEX` command.

Storage: space allocation

- SQL Server uses random access files
- Space allocation in *extents* en *pages*
- Page = 8 kB blok contiguous space
- Extent = 8 logical contiguous pages.
 - uniform extents: for a single db object (table, index, ...)
 - mixed extents: can be shared by 8 database objects
- Each new table or index is allocated in a mixed extent
- If table or index exceeds 8 pages: in uniform extent

Storage: example (db xtreme)

```
CREATE TABLE dbo.SmallRows
(  
    Id int NOT NULL,  
    LastName nchar(60) NOT NULL,  
    FirstName nchar(60) NOT NULL,  
    Title nchar(60) NULL  
);
```

```
INSERT INTO dbo.SmallRows (Id, LastName, FirstName, Title)  
SELECT EmployeeID, LastName, FirstName, Title  
FROM Employee;
```

Storage: example (db xtreme)

```
CREATE TABLE dbo.LargeRows
```

```
(
```

```
    Id int NOT NULL,
```

```
    LastName nchar(600) NOT NULL,
```

```
    FirstName nchar(600) NOT NULL,
```

```
    Title nchar(600) NULL
```

```
);
```

```
INSERT INTO dbo.LargeRows (Id,LastName,FirstName,Title)
```

```
SELECT EmployeeID, LastName, FirstName, Title
```

```
FROM Employee;
```

Storage: example

Undocumented, unsupported feature shows page id for each line in result set:

SQLQuery3.sql...jcor864 (62)*	SQLQuery2.sql...jcor864 (56)*	SQLQuery1.sql...jcor864 (53)*	NB22-DMTVC...ow2010 - ERD	SQLQuery4.sql...jcor864 (64)*																																																																																																																																																								
1	2	3		1	2	3																																																																																																																																																						
<pre>SELECT sys.fn_PhysLocFormatter(%%physloc%%) AS [Row_Locator], Id FROM dbo.SmallRows;</pre>				<pre>SELECT sys.fn_PhysLocFormatter(%%physloc%%) AS [Row_Locator], Id FROM dbo.LargeRows;</pre>																																																																																																																																																								
231% No Issues found Ln: 3 Ch: 23 TABS CRLF Results Messages <table> <thead> <tr> <th></th><th>Row_Locator</th><th>Id</th></tr> </thead> <tbody> <tr><td>1</td><td>(1:11496:0)</td><td>1</td></tr> <tr><td>2</td><td>(1:11496:1)</td><td>2</td></tr> <tr><td>3</td><td>(1:11496:2)</td><td>3</td></tr> <tr><td>4</td><td>(1:11496:3)</td><td>4</td></tr> <tr><td>5</td><td>(1:11496:4)</td><td>5</td></tr> <tr><td>6</td><td>(1:11496:5)</td><td>6</td></tr> <tr><td>7</td><td>(1:11496:6)</td><td>7</td></tr> <tr><td>8</td><td>(1:11496:7)</td><td>8</td></tr> <tr><td>9</td><td>(1:11496:8)</td><td>9</td></tr> <tr><td>10</td><td>(1:11496:9)</td><td>10</td></tr> <tr><td>11</td><td>(1:11496:10)</td><td>11</td></tr> <tr><td>12</td><td>(1:11496:11)</td><td>12</td></tr> <tr><td>13</td><td>(1:11496:12)</td><td>13</td></tr> <tr><td>14</td><td>(1:11496:13)</td><td>14</td></tr> <tr><td>15</td><td>(1:11496:14)</td><td>15</td></tr> <tr><td>16</td><td>(1:11496:15)</td><td>16</td></tr> <tr><td>17</td><td>(1:11496:16)</td><td>21</td></tr> <tr><td>18</td><td>(1:11496:17)</td><td>22</td></tr> <tr><td>19</td><td>(1:11496:18)</td><td>23</td></tr> <tr><td>20</td><td>(1:11496:19)</td><td>24</td></tr> <tr><td>21</td><td>(1:11496:20)</td><td>25</td></tr> <tr><td>22</td><td>(1:11497:0)</td><td>26</td></tr> <tr><td>23</td><td>(1:11497:1)</td><td>27</td></tr> <tr><td>24</td><td>(1:11497:2)</td><td>28</td></tr> </tbody> </table> 8*1024 bytes/page ----- ~ 22 rows/page 4+3*(2*60) bytes/row					Row_Locator	Id	1	(1:11496:0)	1	2	(1:11496:1)	2	3	(1:11496:2)	3	4	(1:11496:3)	4	5	(1:11496:4)	5	6	(1:11496:5)	6	7	(1:11496:6)	7	8	(1:11496:7)	8	9	(1:11496:8)	9	10	(1:11496:9)	10	11	(1:11496:10)	11	12	(1:11496:11)	12	13	(1:11496:12)	13	14	(1:11496:13)	14	15	(1:11496:14)	15	16	(1:11496:15)	16	17	(1:11496:16)	21	18	(1:11496:17)	22	19	(1:11496:18)	23	20	(1:11496:19)	24	21	(1:11496:20)	25	22	(1:11497:0)	26	23	(1:11497:1)	27	24	(1:11497:2)	28	231% No Issues found Ln: 3 Ch: 23 TABS CRLF Results Messages <table> <thead> <tr> <th></th><th>Row_Locator</th><th>Id</th></tr> </thead> <tbody> <tr><td>1</td><td>(1:1488:0)</td><td>1</td></tr> <tr><td>2</td><td>(1:1488:1)</td><td>2</td></tr> <tr><td>3</td><td>(1:1489:0)</td><td>3</td></tr> <tr><td>4</td><td>(1:1489:1)</td><td>4</td></tr> <tr><td>5</td><td>(1:1490:0)</td><td>5</td></tr> <tr><td>6</td><td>(1:1490:1)</td><td>6</td></tr> <tr><td>7</td><td>(1:1491:0)</td><td>7</td></tr> <tr><td>8</td><td>(1:1491:1)</td><td>8</td></tr> <tr><td>9</td><td>(1:1492:0)</td><td>9</td></tr> <tr><td>10</td><td>(1:1492:1)</td><td>10</td></tr> <tr><td>11</td><td>(1:1493:0)</td><td>11</td></tr> <tr><td>12</td><td>(1:1493:1)</td><td>12</td></tr> <tr><td>13</td><td>(1:1494:0)</td><td>13</td></tr> <tr><td>14</td><td>(1:1494:1)</td><td>14</td></tr> <tr><td>15</td><td>(1:1495:0)</td><td>15</td></tr> <tr><td>16</td><td>(1:1495:1)</td><td>16</td></tr> <tr><td>17</td><td>(1:1496:0)</td><td>21</td></tr> <tr><td>18</td><td>(1:1496:1)</td><td>22</td></tr> <tr><td>19</td><td>(1:1497:0)</td><td>23</td></tr> <tr><td>20</td><td>(1:1497:1)</td><td>24</td></tr> <tr><td>21</td><td>(1:1498:0)</td><td>25</td></tr> <tr><td>22</td><td>(1:1498:1)</td><td>26</td></tr> <tr><td>23</td><td>(1:1499:0)</td><td>27</td></tr> <tr><td>24</td><td>(1:1499:1)</td><td>28</td></tr> </tbody> </table> 8*1024 bytes/page ----- ~ 4+3*(2*600) bytes/row				Row_Locator	Id	1	(1:1488:0)	1	2	(1:1488:1)	2	3	(1:1489:0)	3	4	(1:1489:1)	4	5	(1:1490:0)	5	6	(1:1490:1)	6	7	(1:1491:0)	7	8	(1:1491:1)	8	9	(1:1492:0)	9	10	(1:1492:1)	10	11	(1:1493:0)	11	12	(1:1493:1)	12	13	(1:1494:0)	13	14	(1:1494:1)	14	15	(1:1495:0)	15	16	(1:1495:1)	16	17	(1:1496:0)	21	18	(1:1496:1)	22	19	(1:1497:0)	23	20	(1:1497:1)	24	21	(1:1498:0)	25	22	(1:1498:1)	26	23	(1:1499:0)	27	24	(1:1499:1)	28
	Row_Locator	Id																																																																																																																																																										
1	(1:11496:0)	1																																																																																																																																																										
2	(1:11496:1)	2																																																																																																																																																										
3	(1:11496:2)	3																																																																																																																																																										
4	(1:11496:3)	4																																																																																																																																																										
5	(1:11496:4)	5																																																																																																																																																										
6	(1:11496:5)	6																																																																																																																																																										
7	(1:11496:6)	7																																																																																																																																																										
8	(1:11496:7)	8																																																																																																																																																										
9	(1:11496:8)	9																																																																																																																																																										
10	(1:11496:9)	10																																																																																																																																																										
11	(1:11496:10)	11																																																																																																																																																										
12	(1:11496:11)	12																																																																																																																																																										
13	(1:11496:12)	13																																																																																																																																																										
14	(1:11496:13)	14																																																																																																																																																										
15	(1:11496:14)	15																																																																																																																																																										
16	(1:11496:15)	16																																																																																																																																																										
17	(1:11496:16)	21																																																																																																																																																										
18	(1:11496:17)	22																																																																																																																																																										
19	(1:11496:18)	23																																																																																																																																																										
20	(1:11496:19)	24																																																																																																																																																										
21	(1:11496:20)	25																																																																																																																																																										
22	(1:11497:0)	26																																																																																																																																																										
23	(1:11497:1)	27																																																																																																																																																										
24	(1:11497:2)	28																																																																																																																																																										
	Row_Locator	Id																																																																																																																																																										
1	(1:1488:0)	1																																																																																																																																																										
2	(1:1488:1)	2																																																																																																																																																										
3	(1:1489:0)	3																																																																																																																																																										
4	(1:1489:1)	4																																																																																																																																																										
5	(1:1490:0)	5																																																																																																																																																										
6	(1:1490:1)	6																																																																																																																																																										
7	(1:1491:0)	7																																																																																																																																																										
8	(1:1491:1)	8																																																																																																																																																										
9	(1:1492:0)	9																																																																																																																																																										
10	(1:1492:1)	10																																																																																																																																																										
11	(1:1493:0)	11																																																																																																																																																										
12	(1:1493:1)	12																																																																																																																																																										
13	(1:1494:0)	13																																																																																																																																																										
14	(1:1494:1)	14																																																																																																																																																										
15	(1:1495:0)	15																																																																																																																																																										
16	(1:1495:1)	16																																																																																																																																																										
17	(1:1496:0)	21																																																																																																																																																										
18	(1:1496:1)	22																																																																																																																																																										
19	(1:1497:0)	23																																																																																																																																																										
20	(1:1497:1)	24																																																																																																																																																										
21	(1:1498:0)	25																																																																																																																																																										
22	(1:1498:1)	26																																																																																																																																																										
23	(1:1499:0)	27																																																																																																																																																										
24	(1:1499:1)	28																																																																																																																																																										

8*1024 bytes/page
----- ~ 22 rows/page
4+3*(2*60) bytes/row

8*1024 bytes/page
----- ~ 2 rows/page
4+3*(2*600) bytes/row

Storage:conclusions

- As seen before: the amount of IO is the number of pages that are read.
- Despite the fact that storage is cheap the data type has a huge impact on I/O (and query execution time).
- Keep this in mind at design time of your database tables; don't wait till queries get slow.

Partitioning

Use case

- In any time based application (accounting, payroll, time series, ...) years of history are stored but >90% of all queries only involve last days or months.
- Also applicable to categorical data
- Partitioning offers a solution to fetch only necessary data.
- For partitioning we need file groups.

Create table in filegroup

--Disk-Based CREATE TABLE Syntax

CREATE TABLE

```
[ database_name . [ schema_name ] . | schema_name . ] table_name
[ AS FileTable ]
( { <column_definition> | <computed_column_definition>
  | <column_set_definition> | [ <table_constraint> ]
| [ <table_index> ] [ ,...n ] } )
[ ON { partition_scheme_name ( partition_column_name ) | filegroup
  | "default" } ]
[ { TEXTIMAGE_ON { filegroup | "default" } } ]
[ FILESTREAM_ON { partition_scheme_name | filegroup
  | "default" } ]
[ WITH ( <table_option> [ ,...n ] ) ]
[ ; ]
```

Create index in filegroup

```
CREATE [ UNIQUE ] [ CLUSTERED | NONCLUSTERED ] INDEX index_name
    ON <object> ( column [ ASC | DESC ] [ ,...n ] )
    [ INCLUDE ( column_name [ ,...n ] ) ]
    [ WHERE <filter_predicate> ]
    [ WITH ( <relational_index_option> [ ,...n ] ) ]
    [ ON { partition_scheme_name ( column_name )
          | filegroup_name
          | default } ]
    [ FILESTREAM_ON { filestream_filegroup_name | partition_scheme_name | "NULL" } ]
[ ; ]
```

Partitioning

- Partition a table in different rowsets based on a column value, e.g. (but not restricted to) a date related field.
- Assign each partition to a separate file or filegroup.
- Older and rarely queried rows are separated from actual rows.
- Most queries need only actual rows → fewer rows have to be searched.

Partitions: example (db stackoverflow)

Use case: most users only want to see posts with a score of at least 4 or 5.

→ Use score as partitioning field

See script “partitioning.sql”

References

“Effective SQL, 61 specific ways to Write Better SQL”,
Viescas et. al., 2017, Pearson Education

“SQL Performance Explained”, Markus Winand, 2023

“Pro T-SQL 2012 Programmer’s guide, 3d edition”,
Natarjan et. al., 2012, Apress

SQL Server 2017 Query Performance Tuning, Grant Fritchey, 2018, Apress